



IES GABRIEL GARCÍA MÁRQUEZ

TRABAJO FIN DE CICLO

APUNTAME

CICLO FORMATIVO DE GRADO SUPERIOR DE
DAM

INFORMÁTICA Y COMUNICACIONES

3/12/2025

NOMBRE DEL ALUMNO/A: Alberto Pérez Navarro

TUTOR/A DEL TFC: Diego Di Gionantonio

ÍNDICE

1. JUSTIFICACIÓN DEL PROYECTO

2. OBJETIVOS

2.1 Objetivo general

2.2 Objetivos específicos

3. INTRODUCCIÓN

3.1 Angular y TypeScript

3.2 Spring Boot y Java

3.3 PostgreSQL

3.4 JWT y CORS

3.5 WebSockets, STOMP y SockJS

3.6 Docker

3.7 Git y GitHub

3.8 REST

4. METODOLOGÍA Y DESARROLLO DEL PROYECTO

4.1 Flujo general de la aplicación

4.2 Características funcionales

4.2.1 Sistema de autenticación

4.2.2 Navegación

4.2.3 Toma de pedidos

4.2.4 Visualización de pedidos

4.2.5 Edición de pedidos

4.2.6 Panel de administración

4.3 Características técnicas

4.3.1 Seguridad

4.3.1.1 Autenticación JWT

4.3.1.2 Contraseñas

4.3.2 Comunicación en tiempo real

4.3.3 Sincronización tiempo cliente-servidor

4.3.4 API REST

4.3.5 Base de datos

4.3.6 Despliegue

4.3.7 Configuración e instalación

4.3.8 Inicialización de datos

5. RESULTADOS Y DISCUSIÓN

5.1 Funcionamiento general del sistema

5.2 Problemas encontrados y soluciones

6. CONCLUSIONES

6.1 Cumplimiento de Objetivos

6.2 Valoración personal

7. COSTE DESGLOSADO DEL PROYECTO

7.1 Coste de personal

7.2 Coste de software

7.3 Coste de hardware y otros

7.4 Coste total estimado

8. REFERENCIAS BIBLIOGRÁFICAS

1. JUSTIFICACIÓN DEL PROYECTO

Cuento con experiencia trabajando en hostelería y, a lo largo de este tiempo, he podido identificar y sufrir múltiples problemas relacionados con el proceso informatizado de pedidos. Estos inconvenientes afectan tanto a los empleados y su eficiencia como a la experiencia de los clientes, generando errores, retrasos y confusión.

Varios de los problemas más comunes que he observado son:

- Interfaces poco intuitivas, que requieren un largo periodo de adaptación para poder usarlas de manera eficiente.
- Menús desorganizados, donde los ítems no están ubicados de forma lógica.
- Gestión deficiente de la personalización de platos, con falta de listas claras de ingredientes y alérgenos.
- Problemas en la visualización y formato de los pedidos, tickets desestructurados y sin indicadores claros sobre cambios realizados en los ingredientes.
- Mala comunicación entre sistemas, lo que provoca desincronización, pedidos duplicados o pérdida de pedidos en mitad del proceso.
- Falta de herramientas útiles como una búsqueda rápida de productos del menú.

Estos problemas me han llevado a intentar desarrollar una solución más eficiente, o en el proceso, descubrir que estos sistemas tienen una complejidad mayor de la aparente.

“Every single proyect has always started from me being frustated with other people being incompetent... and i said how hard can it be?”

- Linus Torvalds

2. OBJETIVOS

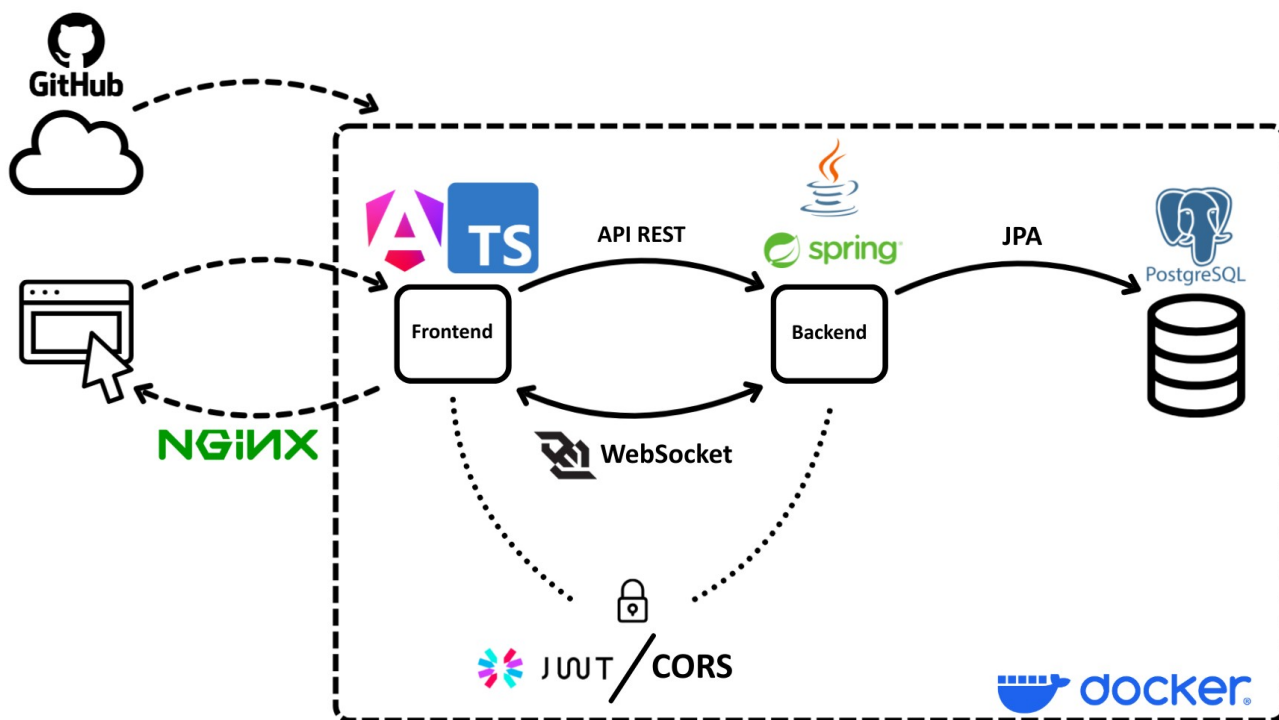
2.1 Objetivo general

Desarrollar un sistema de gestión de pedidos para un restaurante que sea eficiente e intuitivo, permitiendo optimizar el proceso de toma, procesamiento y visualización de pedidos.

2.2 Objetivos específicos

- Desarrollar un sistema completamente local, incluyendo la base de datos, la lógica y la interfaz de usuario, permitiendo su uso sin acceder a internet.
- Crear una interfaz que se ajuste automáticamente a diferentes tamaños de pantalla.
- Incluir un historial de pedidos con información detallada de cada pedido.
- Implementar un sistema de usuarios para restringir accesos.
- Optimizar el flujo de pedidos:
 - Un camarero registra un pedido, se envía al servidor que lo procesará y lo transmite a cocina. Una vez preparado, cocina notificará al servidor y este notificará a los camareros.
- Implementar herramientas para la toma de pedidos:
 - Barra de búsqueda rápida de productos, visualización clara de ingredientes y alérgenos y opciones intuitivas para modificar pedidos.
- Documentar el proceso de desarrollo e implementación del sistema, así como elaborar un manual de usuario.
- Crear un aplicación lo mas accesible posible para usuarios sin conocimientos informáticos (fácil de instalar, configurar y usar).
- Permitir que cada estación de trabajo visualice los productos que le corresponden proporcionando un filtro de “categorías de productos” en la vista de pedidos.

3. INTRODUCCIÓN



(Diagrama de la arquitectura del proyecto)

3.1 Angular y TypeScript

Angular es un framework de desarrollo web basado en componentes. Utiliza TypeScript, un superset de JavaScript con tipado estático que permite detectar errores en tiempo de compilación. Incluye herramientas como un CLI que facilita la creación de proyectos y componentes. Para los estilos se utiliza Angular Material, una biblioteca que proporciona componentes prediseñados, y SCSS, un lenguaje de estilos similar a CSS que permite la anidación de selectores. Los conocimientos de HTML y CSS y el manejo de eventos aprendidos en Lenguajes de Marcas y Desarrollo de Interfaces serán directamente aplicables.

Se eligió Angular porque incluye muchas funcionalidades necesarias para el proyecto (formularios, cliente HTTP) y las que no incluye se integran fácilmente mediante librerías. Además, al ser un framework muy popular, cuenta con abundante documentación y recursos en internet, lo que facilita el desarrollo.

3.2 Spring Boot y Java

Spring Boot es un framework de Java que simplifica la creación de aplicaciones. Incluye librerías para ampliar su funcionalidad como Spring Security para la autenticación, Spring Data JPA para el acceso a la BBDD y Spring WebSocket para la comunicación en tiempo real. Tanto Java como Spring han sido usados durante todo el curso además de Spring Data JPA en acceso a datos.

3.3 PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto, resistente a cargas altas de trabajo. Los conocimientos de SQL y bases de datos relacionales han sido aprendidos en Base de Datos.

Inicialmente se intentó usar SQLite embebido por su simplicidad. Sin embargo, los drivers JDBC de SQLite no proporcionaban la funcionalidad deseada: Hibernate no conseguía crear las tablas y relaciones de manera apropiada. Se acabó por elegir PostgreSQL por su integración nativa con Spring JPA, ser de código abierto y su robustez.

3.4 JWT y CORS

JWT (JSON Web Tokens) es un estándar para transmitir información de manera segura y sin estado (stateless). Cada token se firma digitalmente, garantizando la integridad y autenticidad de la información. Se eligió usar JWT porque se mencionó en clase, lo que despertó mi interés, al investigar descubrí que su naturaleza stateless lo hace ideal para la APIs REST de la aplicación.

CORS (Cross-Origin Resource Sharing) es un mecanismo de seguridad de los navegadores que controla qué dominios pueden hacer peticiones HTTP a un servidor.

3.5 WebSockets, STOMP y SockJS

WebSocket es un protocolo de comunicación que permite tanto al cliente como al servidor enviar mensajes en cualquier momento.

STOMP (Simple Text Oriented Messaging Protocol) es un protocolo de mensajería que permite suscribirse a canales de información para enviar y recibir información.

SocketJS es una biblioteca que proporciona fallbacks automáticos cuando WebSocket no está disponible.

3.6 Docker

Docker es una plataforma que empaqueta aplicaciones junto con sus dependencias en contenedores, garantizando que funcionen siempre igual independientemente del entorno. Se eligió por esta portabilidad, además de facilitar la escalabilidad y abre la opción de usar la aplicación de manera remota si el cliente lo quisiera mediante Kubernetes.

3.7 Git y GitHub

Git es un sistema de control de versiones que permite llevar un seguimiento de todos los cambios realizados en el código. GitHub es una plataforma que proporciona repositorios remotos para almacenar el código. La gestión de repositorios con Git y GitHub han sido aprendidos en Entornos de Desarrollo.

3.8 REST

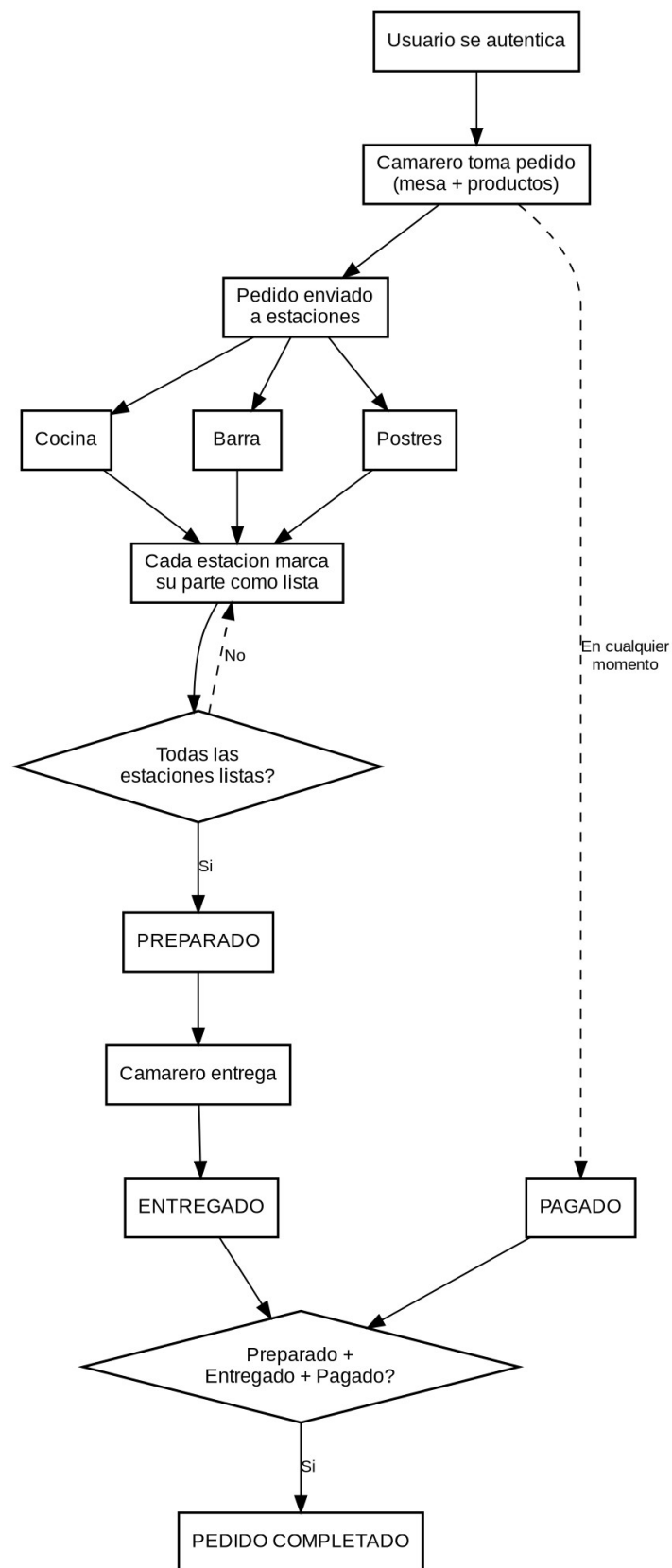
REST (Representational State Transfer) es una arquitectura para diseñar APIs web. Define cómo el cliente y el servidor se comunican mediante peticiones HTTP, utilizando métodos como GET , POST , PUT y DELETE.

4. METODOLOGÍA Y DESARROLLO DEL PROYECTO

4.1 Flujo general de la aplicación

1. Los usuarios se autentican en el sistema
2. El camarero toma un pedido indicando la mesa y los productos
3. El pedido aparece en las pantallas de las estaciones correspondientes (cocina, barra, postres...)
4. Cada estación marca su parte como preparada cuando termina
5. Cuando todas las estaciones terminan, el pedido se marca como preparado

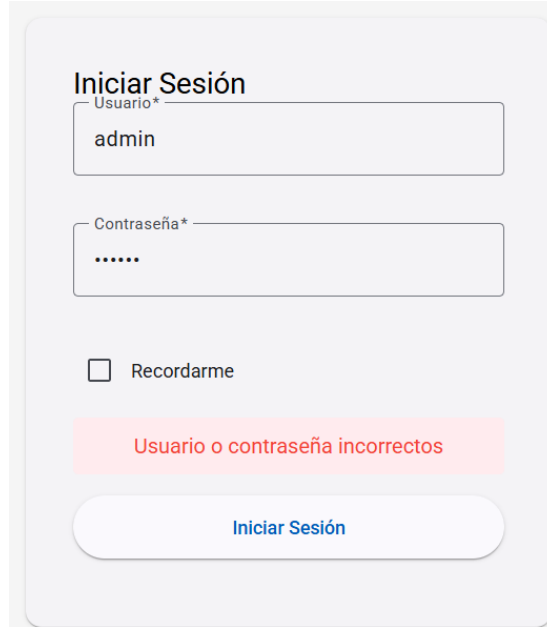
6. El camarero entrega el pedido y lo marca como entregado
7. El pago del pedido puede realizarse en cualquier punto del proceso
8. El pedido se considera completado cuando está preparado, entregado y pagado



(Diagrama de flujo del ciclo de vida de un pedido)

4.2 Características funcionales

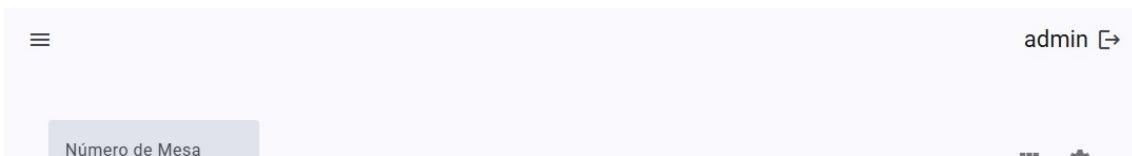
4.2.1 Sistema de autenticación



(Formulario login)

La página de login es la puerta de entrada a la aplicación. Incluye un formulario de login con campos de usuario y contraseña que se validan. Cuando las credenciales son incorrectas, se muestra un mensaje de error. Las páginas protegidas redirigen automáticamente al login si no hay una sesión válida. Dispone de una opción “Recuérdame” que permite mantener la sesión activa.

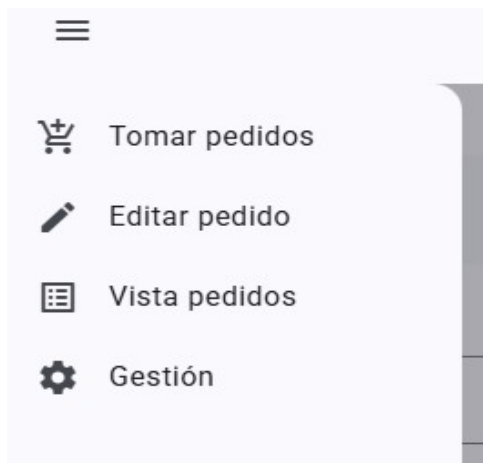
Una vez autenticado, el usuario accede por defecto a la página de tomar pedidos. Una barra superior muestra el nombre del usuario y permite cerrar sesión.



(Top bar)

4.2.2 Navegación

La barra superior incluye un botón de menú que despliega un panel lateral (sidenav) con las diferentes secciones de la aplicación: tomar pedidos, ver pedidos, editar pedidos y gestión. Al seleccionar una opción, el usuario navega a esa sección y el panel se cierra automáticamente evitando ocupar espacio en pantalla.



(Menú navegación)

4.2.3 Toma de pedidos



(Menú toma de pedidos)

La página permite la toma de pedidos de forma rápida e intuitiva, también dispone de un buscador que permite filtrar los productos por nombre.

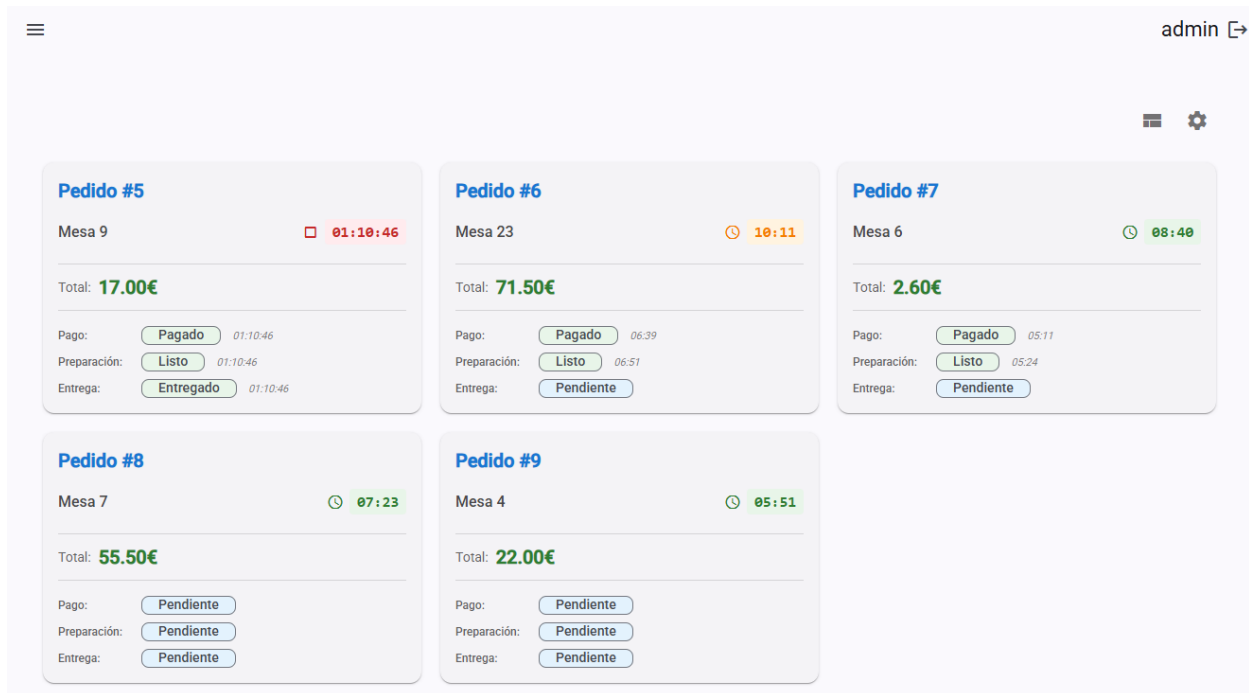
Un panel lateral muestra el resumen del pedido con los artículos añadidos, sus cantidades y el total a pagar. Desde este panel se introduce el número de mesa y se puede modificar la cantidad de cada producto. La interfaz ofrece tres modos de visualización (compacto, normal y grande) para adaptarse a las preferencias de cada usuario y dispositivo. Un botón permite confirmar y enviar el pedido al servidor.

También permite filtrar por categorías mediante un botón que abre un diálogo con las opciones de filtrado. En él se pueden seleccionar las categorías deseadas, buscar categorías por nombre y elegir el modo de filtrado: OR (el producto pertenece a una o más categorías) o AND (el producto debe pertenecer a todas las categorías seleccionadas).



(Diálogo filtrado categorías de productos)

4.2.4 Visualización de pedidos

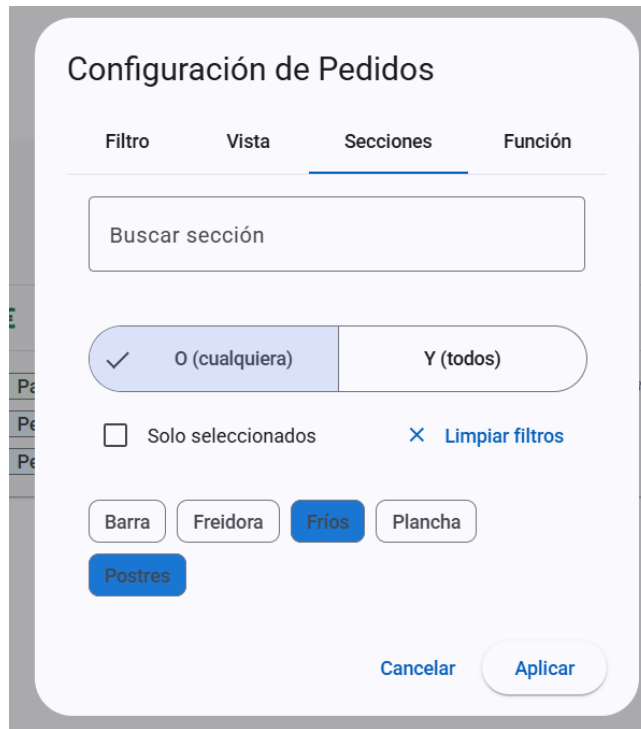


Pedido #	Mesa	Total	Pago	Preparación	Entrega	Estado
Pedido #5	Mesa 9	17.00€	Pagado	Listo	Entregado	01:10:46
Pedido #6	Mesa 23	71.50€	Pagado	Listo	Pendiente	10:11
Pedido #7	Mesa 6	2.60€	Pagado	Listo	Pendiente	08:40
Pedido #8	Mesa 7	55.50€	Pendiente	Pendiente	Pendiente	07:23
Pedido #9	Mesa 4	22.00€	Pendiente	Pendiente	Pendiente	05:51

(Menú vista de pedidos)

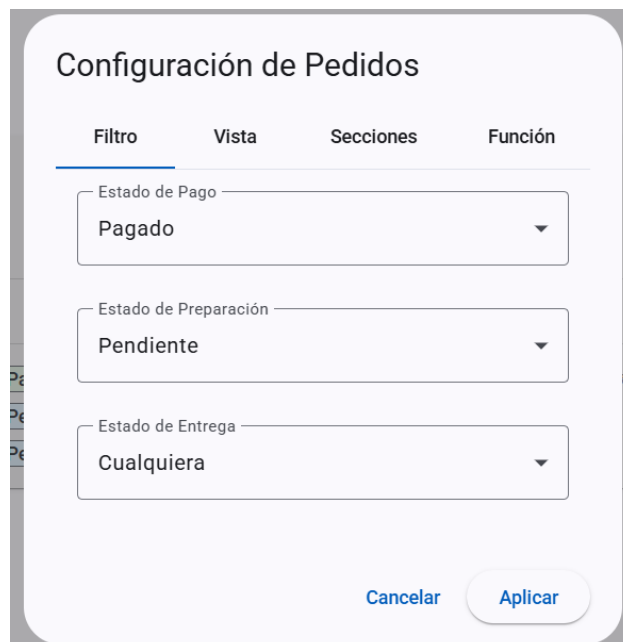
Pantalla principal del sistema, utilizada por todos los miembros del restaurante en algún punto de la vida del pedido. Se actualiza automáticamente en tiempo real gracias a WebSocket sin necesidad de recargar la página. Cada pedido se muestra en una tarjeta que contiene: número de mesa, total del pedido, tiempo transcurrido desde su creación, los tres estados (preparación, pago y entrega) y la lista de productos. El temporizador cambia de color según el tiempo avanza. Al pulsar en un pedido se puede cambiar sus estados de pendiente a completado (pagado, preparado y entregado), cuando un estado llega a completado se muestra el tiempo que ha tardado en conseguirlo. La interfaz ofrece tres modos de visualización (compacto, normal y grande).

Un diálogo de configuración permite personalizar el comportamiento de la página. Se pueden filtrar los pedidos por secciones para que cada estación vea solo los pedidos con los productos que le sean relevantes, seleccionando las secciones deseadas, permitiendo buscar secciones por nombre y eligiendo el modo de filtrado: OR o AND.



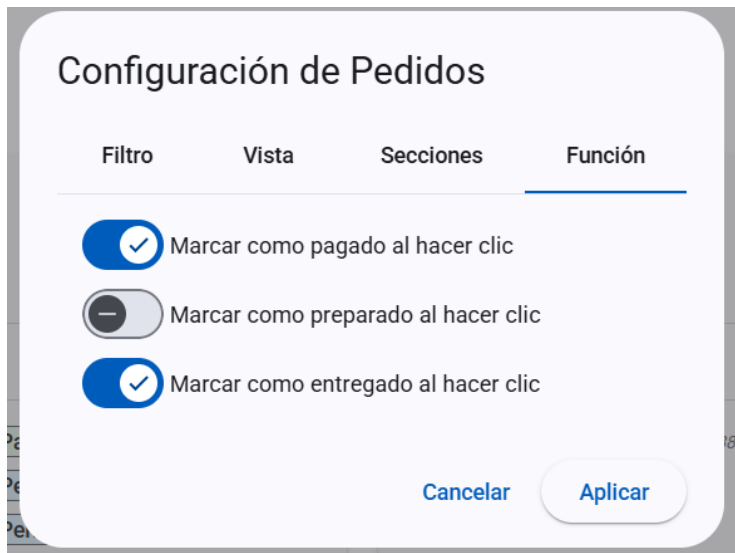
(Diálogo filtrado secciones de productos en pedidos)

También permite filtrar por el valor de los estados, como mostrar solo pedidos pendientes de preparación.



(Diálogo filtrado pedidos por estados)

Las acciones configuran qué ocurre al pulsar un pedido: marcar como entregado, pagado, preparado o cualquier combinación.



Configuración de Pedidos

Filtro Vista Secciones **Función**

☒ Marcar como pagado al hacer clic

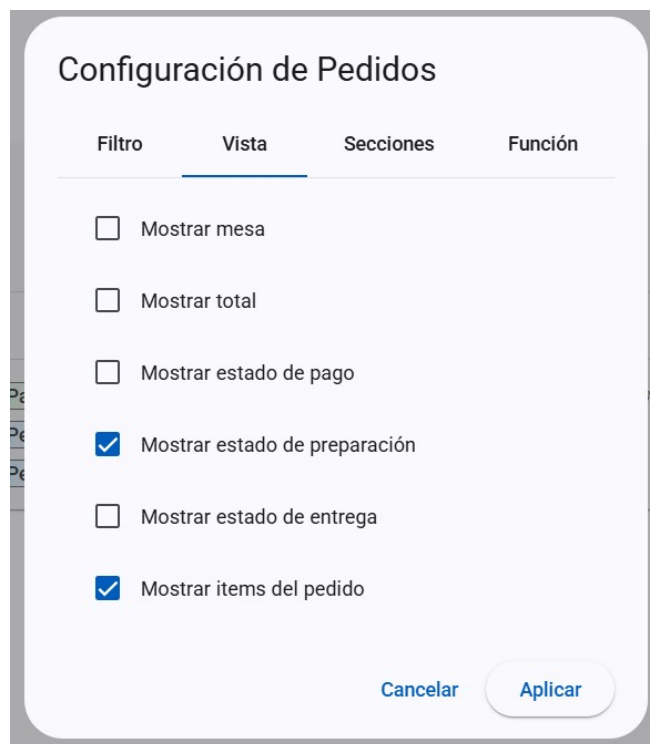
☐ Marcar como preparado al hacer clic

☒ Marcar como entregado al hacer clic

Cancelar Aplicar

(Diálogo selección funcionalidad)

Las opciones de visualización permiten elegir qué campo del pedido se muestra.



Configuración de Pedidos

Filtro **Vista** Secciones Función

☐ Mostrar mesa

☐ Mostrar total

☐ Mostrar estado de pago

☒ Mostrar estado de preparación

☐ Mostrar estado de entrega

☒ Mostrar items del pedido

Cancelar Aplicar

(Diálogo personalización de vista)

4.2.5 Edición de pedidos



ID	Estado de Pago	Estado de Preparación	Estado de Entrega	Mesa	Fecha	Tomado por	Total
77	Pendiente	Pendiente	Pendiente	3	2025-12-03 18:31:44	admin	6.00€
76	Pendiente	Pendiente	Pendiente	15	2025-12-03 18:31:36	admin	4.50€
75	Pendiente	Pendiente	Pendiente	80	2025-12-03 18:31:26	admin	9.80€
74	Pendiente	Pendiente	Pendiente	3	2025-12-03 18:31:23	admin	10.50€
73	Pendiente	Pendiente	Pendiente	12	2025-12-03 18:31:20	admin	12.00€

(Menú edición pedidos)

Funcionalidad orientada a la gestión y modificación de pedidos existentes. Muestra una tabla con todos los pedidos de sistema incluyendo, id, los tres estados del pedido, mesa, fecha de creación, camarero que tomó el pedido y total.

Al pulsar en un pedido se abre un diálogo de edición que muestra todos los artículos del pedido con sus cantidades y precios. Se pueden modificar las cantidades de cada artículo o eliminarlos del pedido. Los cambios no se mandan al backend y se aplican hasta que no se pulsa el botón “Confirmar”.



Pedido #58 - Mesa 13

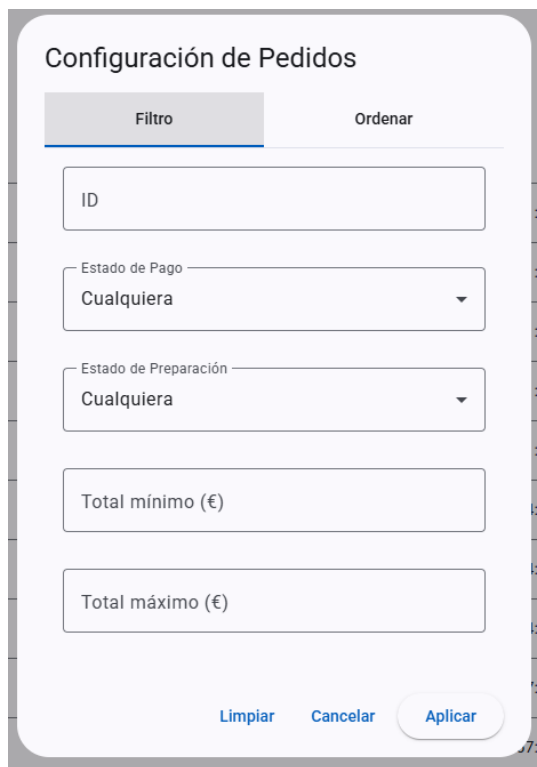
Tostadas	2.50€	—	1	+
Cerveza	2.50€	—	2	+
Croissant	1.80€	—	1	+
Bocadillo jamón	4.50€	—	3	+

Total: 22.80€

[Cancelar](#) [Confirmar](#)

(Diálogo edición pedido)

Un diálogo de configuración proporciona herramientas para facilitar la búsqueda de los pedidos. La pestaña de filtro permite buscar por id del pedido, filtrar por los estados del pedido y establecer un rango de precio máximo y mínimo.



Configuración de Pedidos

Filtro Ordenar

ID

Estado de Pago
Cualquiera

Estado de Preparación
Cualquiera

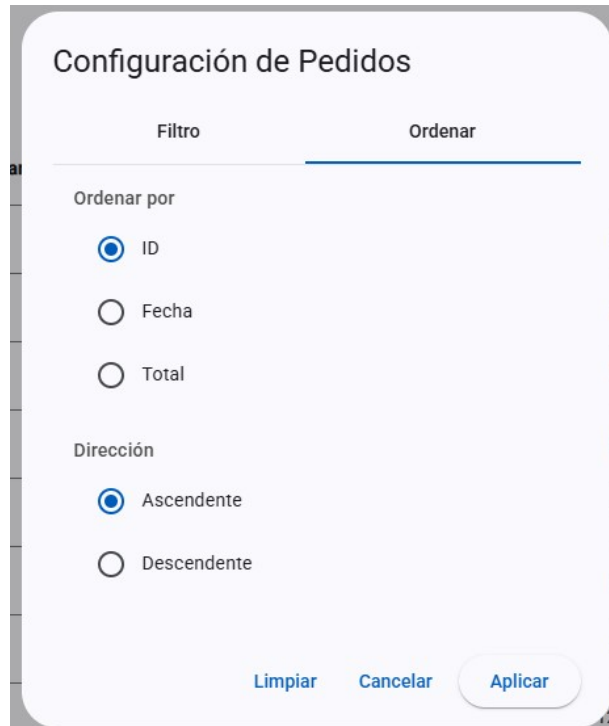
Total mínimo (€)

Total máximo (€)

[Limpiar](#) [Cancelar](#) [Aplicar](#)

(Diálogo filtro búsqueda de pedido)

La pestaña de ordenar permite elegir el criterio de ordenación (id, fecha o total) y si es de manera ascendente o descendente.



The image shows a dialog box titled "Configuración de Pedidos" with two tabs: "Filtro" and "Ordenar". The "Ordenar" tab is selected. Under the heading "Ordenar por", there are three radio button options: "ID" (selected), "Fecha", and "Total". Under the heading "Dirección", there are two radio button options: "Ascendente" (selected) and "Descendente". At the bottom right, there are three buttons: "Limpiar", "Cancelar", and "Aplicar".

(Diálogo orden ordenación búsqueda pedidos)

4.2.6 Panel de administración

admin ↗

Categorías

Secciones

Productos

Usuarios

Buscar

🔍

+ Crear producto

Nombre	Precio	Categorías	Secciones	Acciones
Croquetas de jamón	8.50€	Entrantes	Freidora	 
Jamón ibérico	16.00€	Entrantes	Fríos	 
Tortilla española	6.50€	Entrantes	Plancha	 
Calamares a la romana	10.00€	Entrantes	Freidora	 
Ensalada mixta	7.00€	Ensaladas	Fríos	 

(Menú administración entidades)

La página está organizada en pestañas, cada una dedicada a una entidad: usuarios, artículos, categorías y secciones. La pestaña de usuarios permite gestionar las cuentas del sistema, se pueden crear nuevos usuarios indicando nombre, contraseña y rol, editar el rol de usuarios existente o eliminarlos.

Crear Usuario

Nombre de usuario*

El nombre de usuario es requerido

Contraseña*

La contraseña es requerida

Rol*

Cancelar

Crear

(Diálogo creación usuario)

Editar Usuario

Nombre de usuario*

admin

Rol*

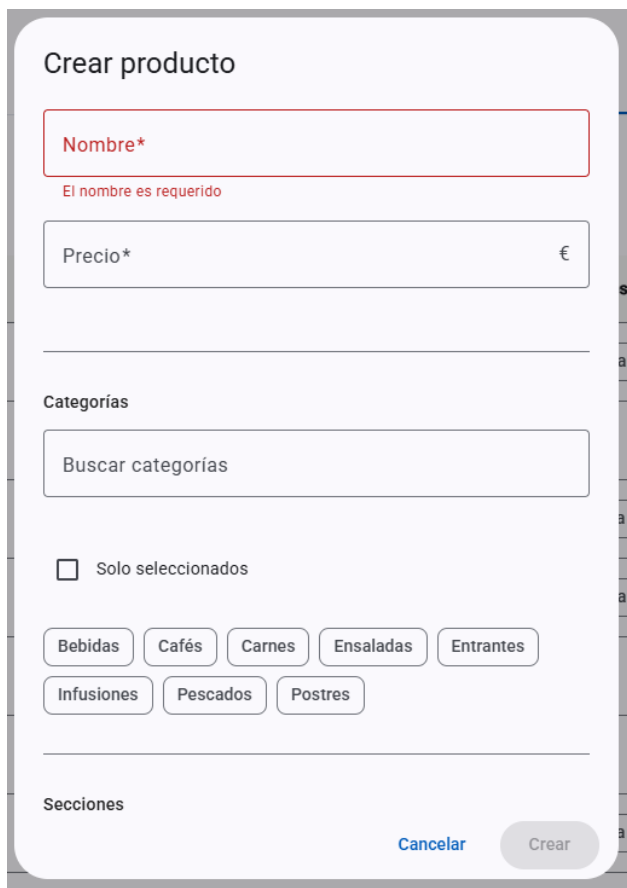
ADMIN

Cancelar

Actualizar

(Diálogo edición usuario)

Al crear o editar un artículo se indica su nombre, precio, y las categorías y secciones a las que pertenece, adicionalmente también se pueden borrar artículos.



Crear producto

Nombre*

El nombre es requerido

Precio* €

Categorías

Buscar categorías

☐ Solo seleccionados

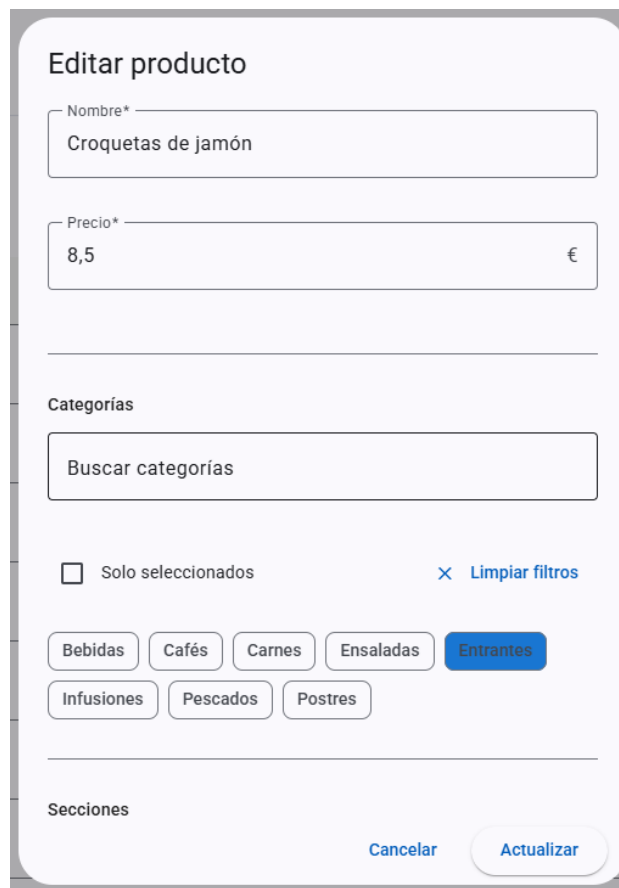
Bebidas Cafés Carnes Ensaladas Entrantes

Infusiones Pescados Postres

Secciones

Cancelar Crear

(Diálogo creación producto)



Editar producto

Nombre*

Croquetas de jamón

Precio* €

8,5

Categorías

Buscar categorías

☐ Solo seleccionados [X Limpiar filtros](#)

Bebidas Cafés Carnes Ensaladas Entrantes

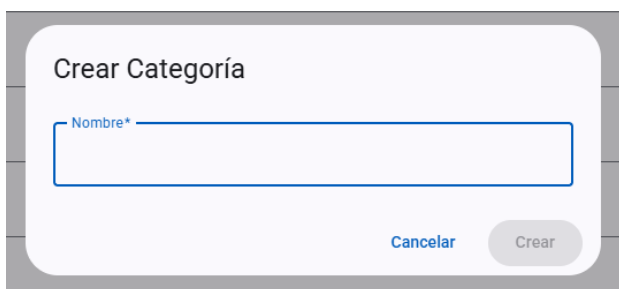
Infusiones Pescados Postres

Secciones

Cancelar Actualizar

(Diálogo edición producto)

Las categorías y secciones solo requieren un nombre, se pueden crear, editar y eliminar.

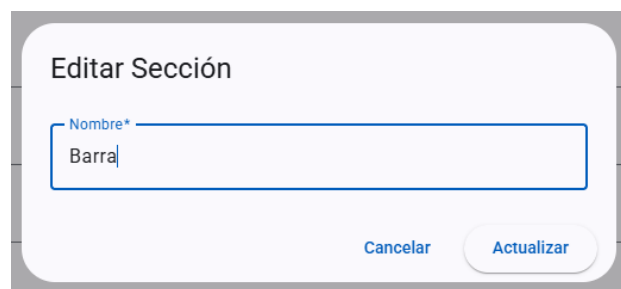


Crear Categoría

Nombre*

Cancelar Crear

(Diálogo creación categoría/sección)



Editar Sección

Nombre*

Barra

Cancelar Actualizar

(Diálogo edición categoría/sección)

4.3 Características técnicas

4.3.1 Seguridad

4.3.1.1 Autenticación JWT

El sistema utiliza tokens JWT que expiran cada 9 horas. Se almacena un único token por usuario en la base de datos, simplificando la gestión frente al enfoque tradicional que mantiene listas de tokens activos y revocados. En cada petición, el servidor compara el token recibido con el almacenado en la base de datos, si no coinciden, la petición se rechaza. Esto permite invalidar un token en cualquier momento simplemente generando uno nuevo que lo reemplace. Al iniciar sesión, si ya existe un token activo en la base de datos, se reutiliza en lugar de generar uno nuevo, esto permite que un mismo usuario acceda desde varios dispositivos simultáneamente compartiendo el mismo token.

En el cliente, dependiendo de la opción “Recuérdame”, el token se guarda en localStorage (memoria persistente del navegador) o sessionStorage (memoria temporal del navegador que borra la información al cerrarlo).

Un interceptor HTTP, que es un componente de Angular que intercepta todas las peticiones antes de enviarlas, añade automáticamente el token en la cabecera de cada solicitud, asegurando que todas las llamadas a la API contengan el token.

Un Guard, que es un mecanismo de Angular que controla el acceso a las rutas, verifica si existe una sesión válida antes de permitir el acceso a páginas protegidas, redirigiendo el login en caso contrario.

4.3.1.2 Contraseñas

Las contraseñas se encriptan con Bcrypt antes de almacenarse en la base de datos, un algoritmo de hash diseñado específicamente para contraseñas que incluye un “salt” automático, esto significa que cada hash será único aunque el texto original sea el mismo, añadiendo una capa adicional de seguridad.

4.3.2 Comunicación en tiempo real

La comunicación en tiempo real es esencial para el funcionamiento de la aplicación, cuando un camarero crea un pedido, este debe aparecer inmediatamente en las pantallas correspondientes. WebSockets con STOMP son perfectos para esto. Los clientes se suscriben al canal “/topic/orders” y reciben automáticamente los eventos de creación y actualización de pedidos. En el caso que el navegador o la red no soporten WebSocket se usa SockJS. Para garantizar la fiabilidad de la conexión, el sistema incluye reconexión automática cada 5 segundos si se pierde la comunicación, permitiendo que los dispositivos recuperen la conexión tras cortes de red sin intervención del usuario, también se envían “heartbeats” cada 4 segundos para mantener la conexión activa, ya que algunas redes cierran conexiones que permanecen inactivas.

4.3.3 Sincronización tiempo cliente-servidor

Durante las pruebas de la aplicación en diferentes dispositivos, se detectó un error, un dispositivo con la fecha y hora mal configuradas mostraba los temporizadores de los pedidos completamente desajustados, el problema era que en el cálculo del tiempo transcurrido desde la creación del pedido se hacía comparando la fecha de creación del pedido (generada por el servidor) con la fecha y hora actual del cliente. Para solucionarlo, se implementó un endpoint que devuelve la fecha y hora actual del servidor. El cliente calcula la diferencia entre la hora del servidor y la suya, la diferencia se aplica a todos los cálculos de tiempo, garantizando que los temporizadores en todos los dispositivos muestren el mismo tiempo transcurrido independientemente de su configuración horaria.

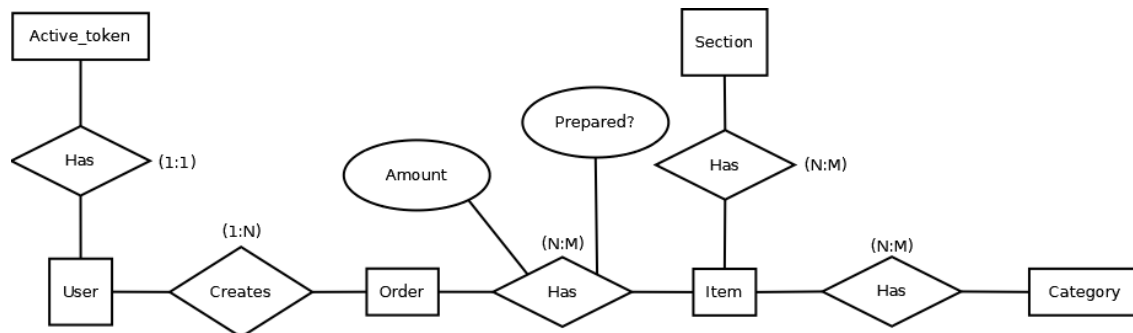
4.3.4 API REST

El backend expone una API REST que permite al frontend realizar operaciones CRUD (crear, leer, actualizar y eliminar) sobre las entidades del sistema.

Para el manejo de errores se implementó un controlador centralizado de excepciones. En lugar de gestionar los errores individualmente en cada endpoint, este controlador intercepta todas las excepciones y devuelve respuestas estructuradas con códigos HTTP apropiados: 404 cuando un recurso no existe, 401 cuando el usuario no está autenticado, 400 cuando los datos enviados son inválidos o 409 al haber conflictos. Esto garantiza que el frontend reciba siempre respuestas consistentes y pueda mostrar mensajes de error claros al usuario.

Se utilizan DTOs (Data Transfer Objects) y anotaciones Jackson para controlar la transferencia de datos entre cliente y servidor. Esto permite definir qué campos son de solo lectura (como las fechas generadas por el servidor), cuáles son solo de escritura (como las contraseñas, que se reciben pero nunca se devuelven) y cuáles se ocultan completamente (como los token de sesión). De esta forma se evita exponer datos sensibles y se reduce el tamaño de las respuestas.

4.3.5 Base de datos



(Diagrama entidad relación)

PostgreSQL almacena las entidades del sistema: User, Order, OrderItem, Item, Category y Section, además de una tabla para los tokens activos. Para optimizar el rendimiento de las consultas se implementaron dos estrategias. Las relaciones entre entidades se definen en los modelos del backend, por ejemplo, el modelo User tiene un campo que referencia todos sus pedidos asociados. El lazy loading (carga perezosa) hace que estas relaciones no se carguen de la base de datos hasta que realmente se necesiten, así, al consultar un usuario no se cargan automáticamente todos sus pedidos, evitando consultas innecesarias. El batch loading agrupa las consultas de relaciones en lotes, cuando se cargan varios pedidos, en lugar de hacer una consulta por cada pedido para obtener sus artículos, se cargan los artículos de todos los pedidos en pocas consultas agrupadas. Las operaciones en cascada garantizan la integridad de los datos, al eliminar un pedido se eliminan sus OrderItems (tabla que almacena qué productos contiene cada pedido) asociados, evitando registros huérfanos en la base de datos.

ActiveToken	
<u>username</u>	PK
token	
expires_at	

(Estructura tabla tokens activos)

User	
<u>id</u>	PK
username	
password	
role	
token	

(Estructura tabla usuario)

La tabla Order ha sido diseñada no solo para gestionar el estado actual de cada pedido, sino también para almacenar información que permita en el futuro implementar funcionalidades de análisis y estadísticas. Además de los campos de estado, registra la fecha y hora de cada fase del ciclo de vida del pedido, así como el usuario que realizó cada una de estas acciones.

Order	
<u>id</u>	PK
table_number	
payment_status	
preparation_status	
delivery_status	
creation_date	
paid_at	
prepared_at	
delivered_at	
taken_by	FK
charged_by	FK
prepared_by	FK
delivered_by	FK

(Estructura tabla pedidos)

- creation_date: momento en que se creó el pedido

- prepared_at: momento en que se marcó como preparado
- delivered_at: momento en que se completó la entrega
- paid_at: momento en que se registró el pago
- taken_by: quien anotó el pedido
- charged_by: quien cobró el pedido
- prepared_by: quien preparó los productos
- delivered_by: quien realizó la entrega

Item	
<u>id</u>	PK
name	
price	

(Estructura tabla productos)

Section	
<u>id</u>	PK
name	

(Estructura tabla secciones)

Category	
<u>id</u>	PK
name	

(Estructura tabla categorías)

Siguiendo el proceso aprendido en clase de diseño de bases de datos relacionales, las relaciones de muchos a muchos (N:M) se han transformado en tablas intermedias. Adicionalmente la tabla intermedia OrderItem tiene dos atributos propios de la relación, amount y prepared.

OrderItem	
<u>order_id</u>	PK,FK
<u>item_id</u>	PK,FK
amount	
prepared	

(Estructura tabla intermedia productos en pedido)

- quantity: cantidad del producto en el pedido
- prepared: producto ya preparado o no

item_category	
<u>item_id</u>	PK,FK
<u>category_id</u>	PK,FK

(Estructura tabla intermedia categorías de producto)

item_section	
<u>item_id</u>	PK,FK
<u>section_id</u>	PK,FK

(Estructura tabla intermedia secciones de producto)

4.3.6 Despliegue

El despliegue se realiza mediante Docker, utilizando contenedores para cada componente del sistema. Se crearon imágenes personalizadas para el frontend y el backend que están publicadas en GitHub Container Registry (ghcr.io), mientras que para la base de datos se utiliza la imagen oficial de PostgreSQL y su configuración se delega a Hibernate, que crea automáticamente las tablas y relaciones a partir de los modelos definidos en el backend. Los Dockerfiles (archivos que definen cómo se construye una imagen Docker) del frontend y el backend utilizan un patrón de construcción en dos etapas. En la primera etapa se compila el código fuente con todas las herramientas necesarias. En la segunda etapa se crea una imagen limpia que solo contiene lo mínimo para ejecutar la aplicación, descartando herramientas como maven, JDK y npm, y reduciendo el tamaño de las imágenes finales. El frontend se sirve mediante Nginx, un servidor web ligero, Angular genera archivos estáticos que necesitan un servidor web para ser entregados al navegador cuando el usuario accede a la URL. Nginx cumple esta función y además está configurado para redirigir todas las rutas a index.html, permitiendo que angular gestione la navegación internamente.

4.3.7 Configuración e instalación

La puesta en marcha es sencilla, instalar Docker, descargar los archivos docker-compose.yml y .env, configurar las credenciales en el archivo .env y ejecutar docker-compose up, lo que descarga automáticamente las imágenes del frontend y backend desde Github Container Registry y la imagen de PostgreSQL desde Docker hub y las prepara para ser usadas. Se intentó integrar en la aplicación una configuración que permitiera establecer una IP o URL de acceso preconfigurada, pero este tipo de configuración debe realizarse siempre a nivel de red, de manera externa a la aplicación y dependerá del cliente.

```
# Configuración Docker de Apuntame
# IMPORTANTE: Reemplaza todos los valores CAMBIAR_ESTO con tus propias credenciales seguras

# Configuración de La base de datos PostgreSQL
POSTGRES_USER=CAMBIAR_ESTO
POSTGRES_PASSWORD=CAMBIAR_ESTO
POSTGRES_DB=apuntame

# Conexión del backend a La base de datos (debe coincidir con Los valores de PostgreSQL anteriores)
DB_HOST=postgres
DB_PORT=5432
DB_NAME=apuntame
DB_USER=CAMBIAR_ESTO
DB_PASSWORD=CAMBIAR_ESTO
```

(credenciales a cambiar en .env)

4.3.8 Inicialización de datos

El sistema incluye un DataInitializer, un componente que se ejecuta automáticamente al arrancar la aplicación por primera vez. Se crea automáticamente un usuario administrador con credenciales por defecto (admin/admin) que permite acceder a la aplicación después de la instalación y está pensado para ser temporal, el cliente debería crear usuarios personalizados y eliminar o modificar las credenciales por seguridad. Adicionalmente, mediante una variable de entorno en el archivo .env se pueden cargar datos de ejemplo facilitando probar el sistema sin necesidad de introducir datos manualmente.

```
# Configuración de datos de demostración
# Establece "true" para cargar datos de ejemplo en el primer arranque, "false" para empezar con la base de datos vacía
INITIALIZE_DEMO_DATA=false
```

(variable en .env para habilitar datos de ejemplo)

5 RESULTADOS Y DISCUSIÓN

5.1 Funcionamiento general del sistema

El sistema funciona correctamente y cumple su propósito principal, gestionar pedidos en tiempo real en un restaurante. Los pedidos se transmiten instantáneamente entre dispositivos y la interfaz permite distinguir la información de manera fácil. La interfaz puede configurarse para mostrar solo la información relevante en cada dispositivo. La aplicación se ejecuta completamente en local sin depender de servicios externos.

5.2 Problemas encontrados y soluciones

- Temporizadores desajustados: Durante las pruebas en diferentes dispositivos, algunos mostraban los temporizadores de los pedidos completamente desajustados. Tras investigar, el problema era que dispositivos con la fecha y hora mal configuradas calculaban el tiempo transcurrido comparando la fecha de creación del pedido (generada por el servidor) con su propia hora local. Se solucionó implementando un endpoint que devuelve la hora del servidor, el cliente calcula la diferencia con su hora local y la aplica a todos los cálculos de tiempo.
- Base de datos: Inicialmente se intentó usar SQLite por su simplicidad, pero los drivers JDBC no proporcionaban la funcionalidad necesaria: Hibernate no conseguía crear las tablas y relaciones correctamente. Se cambió a PostgreSQL por su integración nativa con Spring JPA.
- Finales de línea en scripts: El script de entrada de Docker fallaba sin motivo aparente. Estuve debuggeando pensando que había algo mal con la lógica del script o con Docker. Al ver el mensaje de git sobre CRLF al hacer un commit, se me ocurrió que como Docker ejecuta en un entorno Linux y yo había escrito el script en Windows, los caracteres de retorno de carro (CRLF) podrían estar causando el problema. Tras ejecutar un comando para eliminar los caracteres sobrantes, el script funcionó sin problemas.
- En redes locales el router asigna IPs dinámicas, lo que significa que la IP del servidor puede cambiar tras cada reinicio. La solución es configurar una IP estática en el router, pero esto queda fuera del alcance de la aplicación.
- CORS: CORS (Cross-Origin Resource Sharing) ha sido uno de los problemas más persistentes del proyecto. Al principio ni siquiera conocía su existencia, y cuando empezaron a fallar las peticiones del frontend al backend no entendía por qué. El problema surgió porque el navegador, por seguridad, bloquea las peticiones HTTP a dominios diferentes al de la página actual. En desarrollo, el frontend corría en localhost:4200 y el backend en localhost:8080, lo que el navegador considera orígenes distintos. La primera solución fue crear una configuración de CORS que permitiera peticiones desde localhost y direcciones de red local

(192.168...). Sin embargo, al desplegar con Docker surgieron nuevos problemas: las peticiones desde el puerto 80 (sin especificar puerto en la URL) no coincidían con los patrones configurados que esperaban siempre un puerto explícito. Además, los navegadores envían automáticamente peticiones OPTIONS (llamadas preflight) antes de las peticiones reales para verificar los permisos CORS. Estas peticiones no incluían el token JWT y eran rechazadas por Spring Security. Se tuvo que modificar la configuración de seguridad para permitir explícitamente las peticiones OPTIONS sin autenticación y añadir el filtro CORS antes del filtro de autenticación JWT.

6 CONCLUSIONES

6.1 Cumplimiento de Objetivos

- Sistema local: Funciona correctamente al ejecutar los contenedores de Docker, permitiendo su uso sin conexión a internet.
- UI responsiva: La interfaz se adapta automáticamente según el tamaño de pantalla.
- Historial de pedidos: No existe un historial específico, pero el menú de edición de pedidos cumple esta función además de proporcionar herramientas que facilitan la búsqueda de pedidos.
- Sistema de usuarios: Solo los usuarios dados de alta con un token válido pueden acceder a la aplicación.
- Flujo de pedidos optimizado: El flujo funciona en tiempo real gracias a WebSockets, los pedidos aparecen instantáneamente en las pantallas correspondientes.
- Herramientas para toma de pedidos: Búsqueda de productos por nombre y filtros por categorías.
- Documentación y manual de usuario: No realizado.

- **Aplicación accesible:** La instalación se reduce a configurar el archivo `.env` y ejecutar `docker-compose up`. Existe una limitación con la IP del servidor en redes locales que debe configurarse manualmente. La interfaz usa iconos intuitivos y nombres descriptivos.
- **Filtrado por secciones:** Cada estación puede configurar qué secciones de productos visualiza, mostrando solo los pedidos relevantes.

6.2 Valoración personal

El desarrollo de este proyecto me ha permitido experimentar de primera mano la complejidad que hay detrás de los sistemas de gestión de pedidos que tanto había criticado en mi experiencia laboral.

Aunque los objetivos se han cumplido a grandes rasgos, no todo se ha implementado y no siempre como me lo imaginaba.

Durante el desarrollo se me fueron ocurriendo más ideas de las que he podido implementar. Esto me ha enseñado la importancia de una buena planificación y de definir la relevancia de las tareas. También he descubierto conceptos que desconocía, como CORS, cuya existencia ni siquiera contemplaba y que resultó ser un problema durante el despliegue con Docker.

Los conocimientos de Java y Spring adquiridos durante el ciclo han facilitado la implementación del backend, permitiéndome centrarme más en Angular y el resto de tecnologías con las que estaba menos familiarizado.

7. Coste desglosado del proyecto

Aunque el desarrollo de este proyecto no ha supuesto un desembolso económico directo, es posible realizar una estimación aproximada del coste teórico teniendo en cuenta el valor del tiempo invertido, el uso de herramientas tecnológicas y recursos necesarios para su realización.

7.1 Coste de personal

No he llevado un registro exacto de las horas invertidas en el proyecto, pero considerando el alcance del desarrollo, estimo unas 200 horas de trabajo.

Desarrollo – 200 h – 10€/h – 2000€

7.2 Coste de software

Todas las herramientas utilizadas son gratuitas o de código abierto, lo que ha permitido desarrollar el proyecto sin coste de licencias.

Angular – Coste 0€ - Framework open source

Spring Boot – Coste 0€ - Framework open source

PostgreSQL – Coste 0€ - Base de datos open source

Docker – 0€ - Plataforma open source

Visual Studio Code – 0€ - Editor gratuito

IntelliJ Idea – 0€ - Editor gratuito

Git/Github – 0€ - Control de versiones gratuito

Total 0€

7.3 Coste de hardware y otros

Dado que el proyecto fue desarrollado en mi equipo personal, no se incluye gasto por adquisición de hardware. No obstante, se estima su uso como un coste teórico considerando la amortización del equipo y el consumo eléctrico durante el desarrollo.

Amortización equipo – 44€ - Portátil de 700€, vida útil de 4 años, usado 3 meses

Electricidad – 30€ - Estimación del consumo durante el desarrollo

Total 74€

7.4 Coste total estimado

Personal – 2000€

Software – 0€

Hardware y otros – 74€

Total 2074€

8. Referencias bibliográficas

- Angular: <https://angular.dev/>
- Angular Material: <https://material.angular.dev/>
- Material Icons: <https://fonts.google.com/icons>
- Spring: <https://spring.io/>
- Baeldung: <https://www.baeldung.com/>
- Stack Overflow: <https://stackoverflow.com/>
- Reddit: <https://www.reddit.com/>